

The **ROI Report** Field Guide

Everything you need to start building on the ROI report engine — from the n8n workflow it was born in, through the Claude prompt era, to the agent that runs inside this repo today.

FOR

New teammate · ROI squad

SCOPE

src/lib/roi · /api/roi-agent

READ TIME

~12 minutes

[START HERE](#)

Welcome to the ROI engine 🙌

The ROI report turns a few facts about a prospect's company into a polished, McKinsey-styled financial report — researched, modelled, written and delivered as a PDF. It is the most intricate subsystem in the website repo, and it got here through three distinct lives. Knowing that history is the fastest way to understand *why* the code looks the way it does.



The one-sentence mental model

A user describes their company → an AI agent researches it, builds a deterministic financial model, and writes the report copy → the copy is poured into an HTML template → the template is rendered to a PDF and emailed. Everything else is detail.

Three lives of the ROI report

Same goal every time — a credible ROI report — but the machine behind it was rebuilt twice. Each era solved a real problem and left fingerprints in today's code.

1

The n8n workflow — `ROIGEN (1).json`

ERA ONE · THE ASSEMBLY LINE

The first version lived entirely in **n8n**: a visual, node-by-node pipeline triggered by a webhook. Each step was its own box on a canvas — three chained GPT-4o-mini calls (each locked to a strict JSON schema), with deterministic JavaScript "Code" nodes doing the maths and glue between them.

```
Webhook → Normalize Input → Company Analyst → Parse → ROI Modeler → Parse → ROI Calculator →  
Report Writer → Parse → Assemble Report → HTML Template → Email PDF
```

It worked, but it was fire-and-forget: no web research, no way to edit a report after generation, and the logic was trapped inside a JSON export that no one could code-review or test.

Inheritance → the staged shape

Inheritance → deterministic ROI Calculator

Inheritance → JSON-schema'd LLM calls

2

The Claude template — `template_instructions.txt`

ERA TWO · THE CRAFTSMANSHIP

In parallel, the team ran reports by hand: paste a meeting transcript into Claude alongside a long master prompt — the **"ROI Report Master Instructions v3.2"** — and Claude would produce a finished `.docx`. This era was about *quality*, not automation. The template still defines our standards:

- A silent 6-phase reasoning workflow (intelligence vectoring → financial modelling).
- Financial rules **6A–6E**: sourced blended rates, the 5–20% revenue sanity check, full arithmetic chains, internal consistency cross-checks.
- A locked 17-section report structure and a McKinsey-grade design system (navy cover card, one accent blue, no vertical table borders).
- The vocabulary we still use: "Operational Dividend", "Total Financial Gain", "Hours Returned" — never "cost savings" or "ROI".

Powerful, but fully manual — one skilled person, one prompt, one report at a time.

Inheritance → the financial rulebook

Inheritance → section structure & design tokens

Inheritance → tone & terminology

The in-repo agent — `src/lib/roi/` • today

ERA THREE • THE PRODUCT (YOU ARE HERE)

The current system fuses both: n8n's reliable staged pipeline and the Claude template's rigour — now as real, testable TypeScript inside this Next.js app. The chain of n8n nodes collapsed into a **single unified** `streamText` agent (`src/lib/roi/agent.ts`) that researches, models and writes in one loop, calling *tools* instead of hopping between canvas boxes.

```
POST /api/roi-agent → agent loop { web_search · fetch_page · run_financial_model ·
set_report_copy } → assembleReport → renderTemplate → SSE stream + Supabase + PDF
```

It gained everything the earlier eras lacked: live web research, conversational editing, streaming progress, persistence, and an automated quality eval harness.

Live endpoint · streams SSE

Generate + chat-edit

Researches the web

HOW IT WORKS TODAY

Anatomy of the current system

The whole engine lives under `src/lib/roi/` (TypeScript) and is reached through one route. Here is the map you'll navigate daily.

WHERE	WHAT IT DOES
<code>pages/api/roi-agent.js</code>	The live endpoint. Handles <code>mode: 'generate'</code> and <code>mode: 'chat'</code> , maps the form payload, streams SSE, persists to Supabase. 5-minute serverless timeout.
<code>src/lib/roi/agent.ts</code>	The brain. One <code>streamText</code> loop with ~13 tools that mutate a single <code>ReportState</code> object in place.
<code>pipeline/normalize.ts</code>	Accepts both structured <code>processes[]</code> and legacy flat form fields — the n8n "Normalize Input" node, reborn.
<code>pipeline/roiCalculator.ts</code>	Pure, deterministic financial maths. No LLM. This is the n8n "ROI Calculator" node's direct descendant and enforces template rules 6B–6E.
<code>pipeline/assembleReport.ts</code>	Turns state into every <code>{{ \$json.display.* }}</code> variable — the n8n-style placeholders survived all three eras.
<code>pipeline/renderTemplate.ts</code>	Pours those variables into <code>public/roi-template.html</code> .
<code>prompts/</code>	System prompts for the agent — <code>researchAgent</code> , <code>roiModeler</code> , <code>reportWriter</code> . This is where the template's rulebook now lives as code.
<code>tools/</code>	<code>webSearch</code> (Tavily → Brave → Jina fallback) and <code>fetchPage</code> (Puppeteer).
<code>services/</code>	<code>pdf.ts</code> (Puppeteer + system Chrome), <code>email.ts</code> (Resend), <code>usageTracker.ts</code> .

WHERE	WHAT IT DOES
<code>pages/roi-report.jsx</code> <code>pages/report/[id].jsx</code>	The intake form, and the persisted report viewer / chat editor.
<code>evals/roi/</code>	Scoring harness — quality regression tests for generated reports.



Two gotchas worth knowing on day one

1. `src/lib/roi/` is **TypeScript**; the rest of the repo is JS. The `@components` -style webpack aliases don't work in here — use `@/src/...` imports.
2. If you change an SSE payload in the agent, update its consumer `src/lib/drainSSE.js` too, or the UI goes quiet.

Main features

What the engine can do — each of these is a feature you may be asked to extend.



One-shot generation

`mode: 'generate'` — describe a company, get a full researched report. The agent runs research → model → copy in a single loop.



Conversational editing

`mode: 'chat'` — refine a finished report by chatting. Tools like `update_workflow` and `scale_rates` mutate it live.



Live web research

The agent searches the web for company facts and real salary benchmarks (Glassdoor, Bayt, Gulf Talent...) instead of guessing rates.



Deterministic model

`roiCalculator` does all maths in plain TS — same inputs, same numbers, every time. The financial rulebook is enforced here.



SSE streaming

Progress, partial reports and chat replies stream to the browser in real time via server-sent events.



Persistence & sharing

Every report is saved to Supabase and reopened at `/report/[id]` — viewable and editable later.



PDF + email delivery

`/api/roi-pdf` renders the HTML to PDF;
`/api/roi-email` sends it through Resend.



Bulk mode + evals

`roi/bulk/` turns an Apollo CSV into many reports;
`evals/roi/` scores report quality as a regression test.



Access rules to keep in mind

Non-employees can generate **1 report** and send **5 chat messages**. Employees (role `EMPLOYEE` or an `@lyrise.ai` email) are unlimited. ROI pages assume an authenticated Supabase user.

YOUR TURN

How to add a new feature

Most new work falls into one of four shapes. Find yours, follow the thread end to end, and add an eval case so it stays working.

A A new agent capability → add a tool

Add a `tool({ ... })` in `agent.ts`. Copy an existing one (`update_workflow` is a good template): give it a clear description, a `zod inputSchema`, and an `execute` that mutates `ReportState` in place — never return a new state object. Re-assembly and the SSE update happen for you.

B A new report section → template + assemble

Add the markup and `{{$json.display.yourKey}}` placeholders to `public/roi-template.html`, then produce `yourKey` in `assembleReport.ts`. If the agent needs to write that copy, add a field to `ReportCopy` in `types.ts` and to the `set_report_copy` / `update_copy` tools.

C A new model input → follow the data spine

A new per-workflow number flows through `types.ts` → `roiCalculator.ts` (use it in the maths) → `assembleReport.ts` (display it). A new `form` field flows through `pages/roi-report.jsx` → `mapFormToPayload()` in the API route → `normalize.ts`.

D New behaviour / better output → tune the prompts

Reasoning, tone and rules live in `src/lib/roi/prompts/`. The `template_instructions.txt` master spec is the source of truth for *what* a good report contains — keep prompt edits aligned with it.



Before you open a PR

Run `npm run lint` (the build won't catch lint errors — a Husky pre-commit hook does). Test fast with `devOptions.skipLLM` or `estimatesOnly` to skip slow LLM calls. Add or update a case in `evals/roi/cases/` and run `npm run eval:roi`. Commit only redacted or synthetic client data.

DAY ONE

Get the engine running locally

STEP	COMMAND / NOTE
Node version	Node.js <code>≥ 24</code> is required.
Install & run	<code>npm install</code> then <code>npm run dev</code> .
Env keys	You need <code>OPENAI_API_KEY</code> , a search key (<code>TAVILY_API_KEY</code> or <code>BRAVE_API_KEY</code>), <code>RESEND_API_KEY</code> , and the Supabase keys. <code>.env.example</code> is incomplete for the ROI stack — ask the team for the full set.

STEP	COMMAND / NOTE
Try it	Open <code>/roi-report</code> , sign in, and generate a report. Then open <code>/report/[id]</code> and edit it via chat.
Run an eval	<code>node evals/roi/run.mjs --case <id></code> for one case, or <code>npm run eval:roi</code> for all.
PDF locally	Needs a system Chrome/Chromium install (you have one) unless running on Vercel/Lambda.



Three files to read first

`CLAUDE.md` (repo conventions) · `src/lib/roi/agent.ts` (the loop and its tools) · `template_instructions.txt` (the report rulebook). After those, `roiCalculator.ts` shows you the maths.